

Software-Architektur einer Physik-Simulation in C++

Konrad Wehkamp (5391204) & Lorenz Saalman (5391121)

Zusammenfassung

Die Software-Architektur von GRAVSIM, einer Physik-Simulation in C++, wird in dieser Ausarbeitung vorgestellt. Die Simulation ermöglicht die zeitdiskrete Lösung der Bewegungsgleichungen von Objekten in einem zweidimensionalen Raum, einschließlich Kollisionslösung sowie die Simulation von Kräften wie Gravitation. Strukturell ist der Quellcode modular aufgebaut und folgt einem übergeordneten MVC-Konzept, um eine klare Trennung von Daten, Logik und Darstellung zu gewährleisten. Es werden verschiedene Strukturmuster wie das Beobachter-Muster und das Strategie-Muster verwendet, um die Flexibilität und Erweiterbarkeit der Anwendung zu erhöhen. Anhand von zwei Beispielen wird die konkrete Funktionsweise und Implementierung diskutiert. Zusätzlich wird die autonome Steuerung einer Rakete mit Schubvektorkontrolle simuliert, welche durch drei PID-Regler realisiert wird.

I EINLEITUNG

Simulationen können ein sehr nützliches Werkzeug sein, um komplexe Systeme zu untersuchen und vorherzusagen. Besonders in der Physik und den Ingenieurwissenschaften sind sie ein fester Bestandteil der Forschung und Entwicklung. Sie werden vor allem dann eingesetzt, wenn es schwierig oder unmöglich ist eine analytische Lösung zu finden, oder wenn es unrealistisch ist ein Experiment durchzuführen. Außerdem ermöglichen sie das Durchführen von Designstudien, da sie schnelle Iterationen und Tests von Konzepten möglich machen.

Dieser Bericht beschäftigt sich mit der Software-Architektur einer Physik-Simulation in C++, die die grundlegende Dynamik von Objekten simuliert. Die Simulation löst, beschränkt auf zwei Dimensionen, die Bewegungsgleichungen für eine Vielzahl von Objekten zeitdiskret. Mittels verschiedener Integrationsverfahren kann die Position x_0 und Geschwindigkeit v_0 eines Objekts genutzt werden, um die Position x_1 und Geschwindigkeit v_1 zu einem späteren Zeitpunkt zu approximieren.

Das simple Euler-Verfahren berechnet die Position beispielsweise wie folgt:

$$x_1 = x_0 + v_0 * \Delta t \quad (1)$$

$$v_1 = v_0 + a_0 * \Delta t. \quad (2)$$

Dabei ist a_0 die Beschleunigung zum Zeitpunkt t_0 und Δt die diskrete Zeitschrittgröße. Die Beschleunigung eines Objekts wird durch die Summe aller Kräfte bestimmt

und kann beispielsweise durch Gravitation oder Kollision mit anderen Objekten verursacht werden.

Die Simulation bildet die Grundlage, um das Verhalten und die Steuerung einer Rakete mit Schubvektorkontrolle zu simulieren. Diese soll als Objekt in der Simulation normal teilnehmen, wobei die Schubkraft als zusätzliche Kraft wirkt.

Diese Ausarbeitung beschreibt, wie die Simulation objekt-orientiert umgesetzt wurde und konzentriert sich dabei auf die höhere Struktur und Architektur der Anwendung. Zunächst werden die Ziele der Simulation konkret definiert, bevor die Umsetzung der Software-Architektur detailliert beschrieben wird. Abschließend werden die Ergebnisse und Erkenntnisse aus der Entwicklung der Simulation diskutiert.

II ZIELSETZUNG

Die Zielsetzung der Simulation ist es, eine flexible und erweiterbare Software-Architektur zu entwickeln, die es ermöglicht, verschiedene physikalische Szenarien zu simulieren. Die Position der Objekte soll in einem zweidimensionalen Raum grafisch visualisiert werden.

Folgende Anforderungen wurden definiert:

- Bewegung von Objekten nach newtonischen Bewegungsgleichungen
- Kollisionserkennung und Lösung mit kreisförmigen Objekten
- Krafteinwirkung auf ein Objekt an beliebiger Position
- globale und von massebehaftenden Objekten ausgehende Gravitationskraft
- Einstellung der Simulation durch den Nutzer (z.B. Zeitschrittgröße, Anzahl und Position der Objekte, etc.)
- Implementierung verschiedener Integrationsverfahren

Zusätzlich soll die autonome Steuerung einer Rakete mit Schubvektorkontrolle simuliert werden. Hierfür wurden folgende Ziele definiert:

- Implementierung einer Rakete mit Schubvektorkontrolle als Objekt in der Simulation
- Implementierung eines Regelalgorithmus, der die Schubvektorkontrolle der Rakete ermöglicht

- Definition von Zielpunkten für den Regelalgorithmus

III UMSETZUNG

Die Implementierung erfolgte in C++ unter Verwendung von objekt-orientierten Prinzipien. Der gesamte Code wurde eigenständig geschrieben, welche Bibliotheken verwendet wurden, ist in Tabelle 1 im Anhang vermerkt.

Die Umsetzung der grafischen Oberfläche erfolgt mit GLFW und OpenGL, wobei die Benutzeroberfläche ImGui verwendet. Der Code, sowie die gesamte Entwicklungshistorie können im GitHub Repository ¹ eingesehen werden.

III.I HERANGEHENSWEISE

Um möglichst wenig fehleranfälligen und gut erweiterbaren Code zu schreiben, wurden verschiedene Ansätze verfolgt. Grundsätzlich wurde das Projekt in möglichst unabhängige Module aufgeteilt, wie z.B. die Physik-Engine, die grafische Darstellung, die Benutzeroberfläche, etc. Zwischen den Modulen wurden möglichst kompakte Schnittstellen definiert. Einer der wichtigsten Aspekte war die Trennung von Daten, Logik und Darstellung. Wo immer möglich, wurden Abhängigkeiten so gesetzt, dass die übergeordnete Logik keine Annahmen über die Details der Implementierung der Module treffen muss. So konnte vermehrt Gebrauch vom Abhängigkeits-Inversions-Prinzip und dem Strategie-Muster gemacht werden.

Mit der manuellen Speicherverwaltung in C++ kommt auch die Verantwortung für die korrekte Verwaltung von Ressourcen einher. Fehleranfällig ist vor allem das Anlegen und Freigeben von Objekten im dynamischen Speicher (Heap), hier können leicht ungültige Zeiger, Speicherlecks oder doppelte Freigaben entstehen. Um diese Fehlerquellen zu minimieren, wurde konsequent der Besitz von Variablen mit dem Ownership-Modell festgelegt, welches C++ durch `unique_ptr` ermöglicht. Mit einem `unique_ptr` weist man dem Objekt alleinigen Besitz eines Zeigers zu. Durch den gelöschten Kopier-Konstruktor ist es nicht möglich den Zeiger zu kopieren und er kann lediglich übergeben werden und der Speicher wird automatisch freigegeben, wenn der Zeiger aus dem Gültigkeitsbereich fällt. Ein weiterer Vorteil dieser Zeiger ist es, dass man Intent einer Variable klar kommunizieren kann, das macht den Code verständlicher.

III.II STRUKTUR

Die grundlegende Struktur der Klassen im Projekt ist in Abbildung 1 dargestellt. Der namespace `Core` beinhaltet die Klasse `Application`, welche die Hauptschleife ausführt und für einen `vector` von der Klasse `AppLayer` deren Ereignisfunktionen `OnInit`, `OnUpdate` und `OnRender` aufruft. Außerdem verwaltet sie ein Ereignis-Beobachter-System, mit dem die

`AppLayer` Ereignisse abonnieren und auslösen kann. Der Code in dem namespace `Core` ist auf einem Open-Source Git-Repository ² basiert und wurde für diese Anwendung angepasst.

Der Rest der Anwendung befindet sich im namespace `App` und besteht aus vier Klassen die `AppLayer` erweitern. Die Klasse `EngineLayer` bildet die Basis und verwaltet die Physik-Simulation, das Rendering und die Daten. Die Klasse `UILayer` stellt die Benutzeroberfläche dar, mit der die Simulation gesteuert werden kann. Zusätzlich gibt es den `AudioLayer`, welcher die Wiedergabe von Audio ermöglicht, und den `ControlLayer`, welcher die Steuerung einer Rakete in der Simulation übernimmt. Der `EngineLayer` hängt dabei überhaupt nicht von den anderen `AppLayers` ab.

Die Daten der Simulation werden in der Klasse `Scene` verwaltet, welche einen `vector` von `SceneObject` enthält. Diese Objekte präsentieren physische Objekte in der Simulation und speichern eine Struktur `Transform`, die Position, Rotation und Größe festlegt, sowie Felder für Geschwindigkeit, Masse und andere vom Nutzer definierte Eigenschaften. Außerdem hat jedes Objekt ein `IVisual`, eine Schnittstelle um Daten zur grafischen Darstellung zu speichern und eine Liste von `ColliderBase` für die Kollisionslösung.

Die Struktur verwendet ein übergeordnetes modulares MVC-Konzept, in dem die Model-, View- und Controller-Funktionen auf mehrere Klassen verteilt sind, die gemeinsam die jeweilige Schicht bilden. Das Model besteht aus der Klasse `EngineLayer` und den im vorherigen Abschnitt genannten Datenklassen. Der View wird durch alle Klassen des Rendering Moduls gebildet, welche nicht im Detail in dieser Ausarbeitung beschrieben werden, aber im Anhang in Abbildung 8 als UML-Diagramm dargestellt werden. Dort wird grafische Darstellung der Simulation über OpenGL Shader umgesetzt. Der Controller besteht aus der `UILayer`, welche die Benutzeroberfläche darstellt, und der `ControlLayer`, welche die Steuerung einer Rakete ermöglicht. Abhängigkeiten zwischen den Ebenen sind dabei so umgesetzt, dass `EngineLayer` unabhängig von den anderen Ebenen ist, während `UILayer` und `ControlLayer` von `EngineLayer` abhängen, um die Daten im Modell zu verändern.

Die Physik-Simulation selbst ist in der Klasse `PhysicsSolver` implementiert, welche von `EngineLayer` verwaltet wird. Über eine Schnittstelle `IPropagator` werden verschiedene Integrationsverfahren implementiert, welche von `PhysicsSolver` genutzt werden können. Abbildung 2 zeigt die Struktur und konkreten Klassen der Physik-Simulation. Für die Randbedingungen der Simulation besitzt die Klasse `Scene` eine Liste von `Constraint` und für die Lösung von Kollisionen werden die Kontakte zwischen

¹<https://github.com/grav-byte/GravSim>

²<https://github.com/TheCherno/Architecture>

Objekten in SceneObject als ContactPoint gespeichert, welche von der Klasse ContactSolver verwaltet werden. Im Moment werden nur kreisförmige CircleCollider unterstützt, die Architektur ermöglicht aber die einfache Erweiterung um weitere Kollisionsformen, solange diese ColliderBase erweitern.

III.III PROGRAMMABLAUF

Um die Funktionsweise der Applikation zu verdeutlichen, wird hier der Ablauf eines Schrittes der Physik-Simulation beschrieben. Dieser beginnt in der Hauptschleife in der Klasse App. Zunächst wird die Zeit `simulationDeltaTime` seit dem letzten Schritt berechnet. Dann wird für jede AppLayer die Funktion `OnUpdate` aufgerufen:

```
// main layer update
for (const std::unique_ptr<AppLayer>& layer : layerStack_)
    layer->OnUpdate(simulationDeltaTime);
```

Die Simulation wird über die `EngineLayer` aktualisiert, die einen `PhysicsSolver` besitzt. Diesem wird die aktuelle Szene und `deltaTime` übergeben, wenn die Simulation läuft und nicht pausiert ist:

```
void EngineLayer::OnUpdate(const float deltaTime) {
    if (runningSimulation_ && !pausedSimulation_)
        physicsSolver->UpdatePhysics(scene_.get(),
        ↪ deltaTime);
    ...
}
```

Da der Zeitschritt Δt , mit dem die Simulation läuft, unabhängig von der Bildrate sein soll, wird die Simulation in Zwischenschritten aktualisiert. Dafür wird die vergangene Zeit seit dem letzten Schritt in einem Akkumulator gespeichert und solange die Simulation aktualisiert, bis der Akkumulator kleiner als die Zeitschrittgröße ist:

```
void PhysicsSolver::UpdatePhysics(const Scene* scene, const
    ↪ float deltaTime) {
    ...
    // sub step the physics updates as often as necessary
    while (timeAccumulator_ >= timeStep_) {
        StepPropagation(scene);
        timeAccumulator_ -= timeStep_;
    }
    ...
}
```

In einem Physikschritt muss die Propagation der Objekte und die Kollisionslösung durchgeführt werden. Dafür wird zunächst der `ContactSolver` verwendet, um die Kontakte zwischen den Objekten zu finden. Die Informationen der Kontakte werden dabei in den `SceneObjects` gespeichert und nicht direkt zurückgegeben. Dann wird für jedes Objekt die Propagation durchgeführt, welche die Position und Geschwindigkeit des Objekts aktualisiert. Hier wird keine Annahme über das Integrationsverfahren getroffen, da die Propagation über eine `IPropagator` Schnittstelle integriert ist. Danach werden die `Constraints` angewendet, sie bilden die Randbedingungen der Simulation ab. Zuletzt werden die Kontakte zwischen Objekten gelöst.

```
void PhysicsSolver::StepPropagation(const Scene *scene) const {
    // find contacts
    ContactSolver::ClearContacts(scene);
    ContactSolver::FindContacts(scene);

    // create context for propagation
    const PhysicsContext context{ *scene, *this };
}
```

```
for (SceneObject* object : scene->GetAllObjects()) {
    // propagate object
    activePropagator->Propagate(*object, context, timeStep_);

    if (object->mass > 0.0f) {
        // apply constraints and resolve contacts for objects with mass
        for (const Constraint* constraint : scene->GetConstraints()) {
            constraint->ApplyConstraint(object);
        }
        ContactSolver::ResolveContacts(object);
    }
}
```

Dem `activePropagator` ist es möglich über einen `PhysicsContext` Zugriff auf die Szene und dem `PhysicsSolver` zu haben. So kann die Implementation der Propagation selbst bestimmen, wann und wie oft sie Kräfte mit der Methode `PhysicsSolver::GetAccelerationForObject` berechnen muss. Abbildung 4 zeigt beispielhaft den Call-Stack für die Berechnung von Kräften während der Propagation eines Objekts.

Die Interaktion mit der Simulation erfolgt einzig durch die Veränderung der Daten der Modell-Ebene. So kann die Simulation vollständig entkoppelt laufen, was ihren simplen Aufbau ermöglicht. Als weiteres Beispiel soll das Laden einer Szene dieses Szenario demonstrieren. Wenn der Nutzer auf den Button "Load Scene" klickt, wird folgender Code ausgeführt, welcher den Pfad der ausgewählten Szene an die Methode `EngineLayer::LoadScene` übergibt:

```
if (ImGui::Button("Load Scene")) {
    if (engine->LoadScene(sceneSelector->GetSelectedFile()))
        ShowStatusMessage("Scene loaded successfully.");
    ...
}
```

Die Methode `EngineLayer::LoadScene` stoppt zunächst die Simulation, falls sie läuft, und versucht dann die Szene über die `Serialiser::LoadScene` Methode zu laden. Falls erfolgreich, wird die geladene Szene in der `EngineLayer` in die `unique_ptr<Scene>` geschoben. Sie gehört jetzt `EngineLayer`. Danach wird die Methode `OnSceneLoaded` aufgerufen:

```
bool EngineLayer::LoadScene(const std::string &filePath) {
    if (runningSimulation_)
        StopSimulation();
    ...

    auto loadedScene = Serialiser::LoadScene(filePath);
    if (!loadedScene)
        return false;

    scene_ = std::move(loadedScene);
    OnSceneLoaded();
    return true;
}
```

Die Methode `OnSceneLoaded` verwaltet ihre Abhängigkeiten, wie z.B. das Rendering-Modul. Um andere Ebenen über die neue Szene zu informieren, wird `SceneLoadedEvent`, ein Ereignis, ausgelöst. Dafür wird die Methode `Application::RaiseEvent` über ein Singleton-Muster aufgerufen:

```
void EngineLayer::OnSceneLoaded() const {
    renderingSystem->SetActiveCamera(scene->GetCamera());

    SceneLoadedEvent event(scene_.get());
    Core::Application::Get().RaiseEvent(event);
    ...
}
```

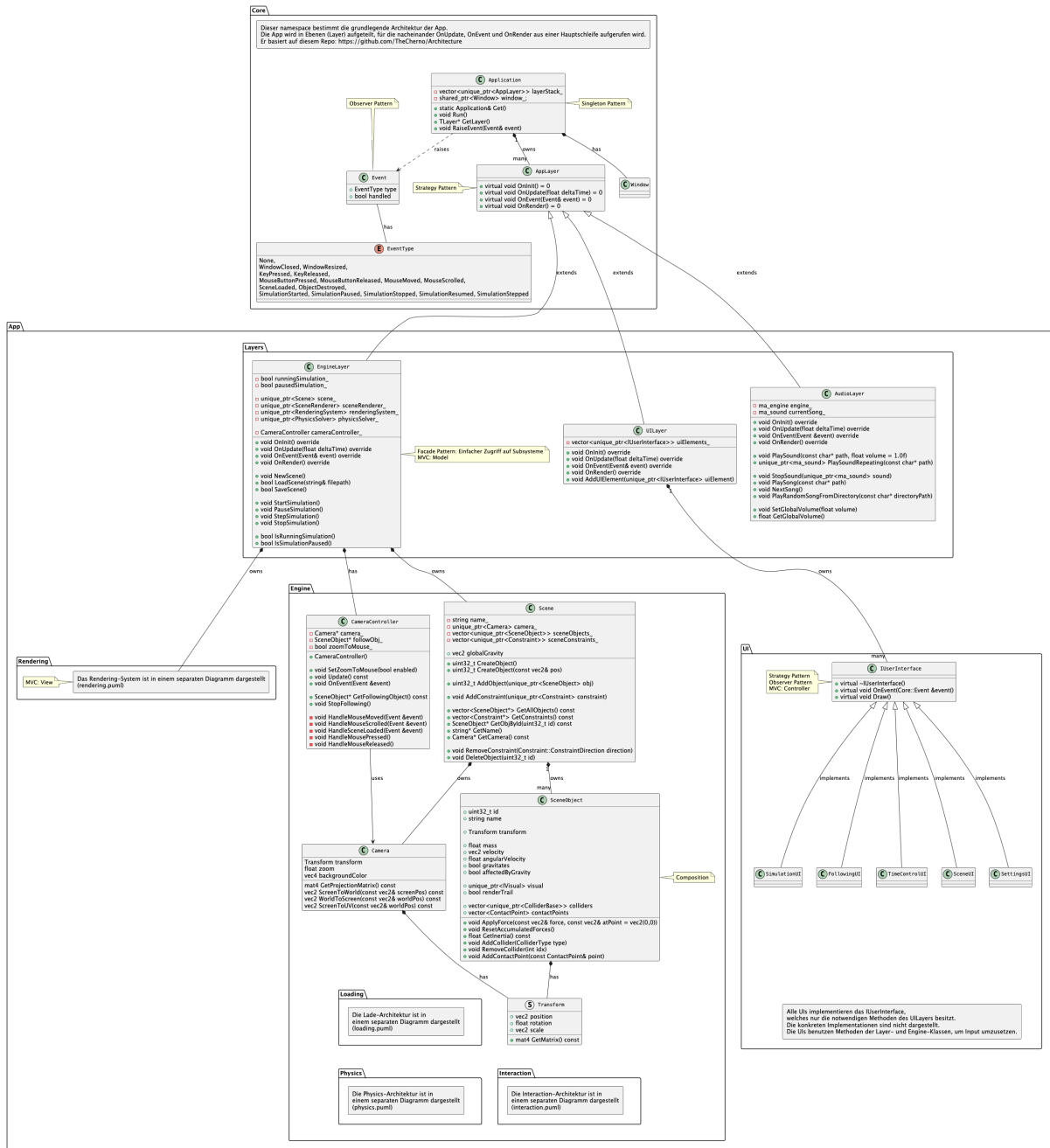


Abbildung 1: Überblick über die oberste Struktur der Anwendung

[illegible]

Abbildung 3: Überblick über die Struktur der Raketensteuerung, welche von der EngineLayer verwaltet wird

```

PhysicsSolver::GetAccelerationForObject(const Scene &, const SceneObject &) const PhysicsSolver.cpp:79
PhysicsContext::GetAcceleration(const SceneObject &) const PhysicsContext.cpp:5
EulerPropagator::Propagate(SceneObject &, const PhysicsContext &, float) EulerPropagator.cpp:9
PhysicsSolver::StepPropagation(const Scene *) const PhysicsSolver.cpp:60
PhysicsSolver::UpdatePhysics(const Scene *, float) PhysicsSolver.cpp:113
EngineLayer::OnUpdate(float) EngineLayer.cpp:169
Core::Application::Run() Application.cpp:88
main main.cpp:38

```

Abbildung 4: Aufrufstapel für die Berechnung von Kräften während der Propagation eines Objekts, dieser zeigt die verschachtelten Aufrufe von Methoden bis zur Berechnung der auf ein Objekt wirkenden Kräfte.

```

SceneUI::OnEvent(Core::Event &) SceneUI.cpp:38
UILayer::OnEvent(Core::Event &) UILayer.cpp:100
Core::Application::RaiseEvent(Core::Event &) const Application.cpp:114
EngineLayer::OnSceneLoaded() const EngineLayer.cpp:74
EngineLayer::LoadScene(const std::string &) EngineLayer.cpp:57
SceneUI::DrawSceneLoading() SceneUI.cpp:79
SceneUI::Draw() SceneUI.cpp:59
UILayer::OnRender() UILayer.cpp:129
Core::Application::Run() Application.cpp:92
main main.cpp:38

```

Abbildung 5: Aufrufstapel für das Laden einer Szene, dieser zeigt die verschachtelten Aufrufe von Methoden bis zum Laden der Szene und der Aktualisierung der Benutzeroberfläche.

Die UILayer und andere Module können dieses Ereignis abonnieren, um über die neue Szene informiert zu werden. So kann beispielsweise die UILayer die Informationen der neuen Szene nutzen, um die Benutzeroberfläche entsprechend zu aktualisieren:

```

void SceneUI::OnEvent(Core::Event &event) {
    // Listen for scene loaded events to update the scene
    ↳ pointer
    if (event.GetEventType() == Core::SceneLoaded) {
        scene_ = engine_>GetScene();
    }
}

```

Der Aufrufstapel für das Laden einer Szene ist in Abbildung 5 dargestellt.

III.IV VERWENDETE STRUKTORMUSTER UND ENTWURFSENTSCHEIDUNGEN

Neben der rein funktionalen Umsetzung wurden bei der Entwicklung der Simulation auch mehrere bewusste Strukturentscheidungen getroffen, um die Wartbarkeit, Erweiterbarkeit und Verständlichkeit des Codes zu verbessern. Dabei kamen sowohl etablierte Strukturmuster als auch projektspezifische Entwurfsentscheidungen zum Einsatz. Ein zentrales Strukturprinzip der Anwendung ist das im vorherigen Abschnitt beschriebene modulare MVC-Konzept, welches die Verantwortlichkeiten der Simulation klar voneinander trennt. Im Folgenden werden weitere Strukturmuster und konkrete Entwurfsentscheidungen erläutert, die in der Simulation verwendet wurden.

Strategy Pattern

Das *Strategy Pattern* wird in der Physik-Simulation bei der Wahl des Integrationsverfahrens eingesetzt. Die Klasse `PhysicsSolver` übernimmt dabei die Rolle des Kontexts, während `IPropagator` die gemeinsame Schnittstelle für konkrete Integrationsverfahren definiert. In `PhysicsSolver::StepPropagation(...)` wird die Propagation eines Objekts über den aktuell gesetzten Strategietyp mittels `activePropagator_>Propagate(...)` ausgeführt. Dadurch kann das numerische Integrationsverfahren ausgetauscht werden, ohne den Ablauf der Physik-Simulation im `PhysicsSolver` ändern zu müssen. Dies ermöglicht insbesondere den direkten Vergleich verschiedener Integrationsverfahren innerhalb derselben Simulationsumgebung.

Observer Pattern

Das *Observer Pattern* wird im Ereignissystem der Anwendung konkret durch die Klasse `Application` umgesetzt, welche Ereignisse zentral verteilt. Ein Beispiel ist das Laden einer Szene: Nachdem `EngineLayer::LoadScene(...)` eine neue Szene übernommen hat, erzeugt `EngineLayer::OnSceneLoaded()` ein `SceneLoadedEvent` und gibt dieses über `Application::RaiseEvent(...)` an das System weiter. Komponenten wie `SceneUI` können dieses Ereignis in `OnEvent(...)` abonnieren und darauf reagieren, ohne dass `EngineLayer` die konkreten Empfänger kennen muss. Dadurch entsteht ein lose gekoppelter, ereignisbasierter Informationsfluss, der die Erweiterbarkeit um zusätzliche Komponenten erleichtert.

Ownership- und Referenzmodell

Das Ownership-Modell der Anwendung trennt klar zwischen Besitz und Zugriff. Der `EngineLayer` besitzt die aktuelle Szene exklusiv über einen `std::unique_ptr<Scene>`, was beim Laden einer Szene durch die Zuweisung `scene_ = std::move(loadedScene);` sichtbar wird. Andere Komponenten erhalten dagegen nur nicht-besitzende Verweise, beispielsweise über `scene_.get()` beim Erzeugen eines `SceneLoadedEvent`. Raw Pointer dienen hier nicht zur Besitzverwaltung, sondern ausschließlich als nicht-besitzende Referenzen auf Objekte mit eindeutig definierter Lebensdauer. Dadurch werden zirkuläre Besitzverhältnisse vermieden und die Speicherverwaltung bleibt zentralisiert und nachvollziehbar.

Abhängigkeitsrichtung der Layer

Die Richtung der Abhängigkeiten ist bewusst so gewählt, dass der Kern der Anwendung unabhängig bleibt. `EngineLayer` kapselt Modell, Physik und rendering-nahe Datenhaltung, ohne von `UILayer` oder `ControlLayer` abhängig zu sein. Umgekehrt greifen

UILayer und ControllLayer auf die Engine zu, um den Zustand der Simulation zu lesen oder zu verändern. Diese einseitige Abhängigkeitsrichtung entspricht dem Prinzip der Abhängigkeitsinversion und verhindert, dass UI- oder steuerungsspezifische Logik in den Simulationskern eindringt. Dadurch wird die langfristige Wartbarkeit und Erweiterbarkeit der Anwendung deutlich verbessert.

III.V REGELALGORITHMUS

Grundsätzlich wird zwischen der Regelung der Schubstärke und der Schubrichtung unterschieden. Für die Schubstärke dient der vertikale Abstand zum Zielpunkt als Fehlergröße, wodurch die Rakete ihre Höhe relativ zum Ziel anpasst. Parallel dazu wird aus der horizontalen Abweichung zunächst eine gewünschte Neigung der Rakete berechnet. Diese stellt einen Zwischensollwert dar, der von einem zweiten Regelkreis verarbeitet wird. Ein dritter Regler sorgt schließlich dafür, dass die tatsächliche Orientierung der Rakete dieser gewünschten Neigung folgt, indem der Winkel des schwenkbaren Triebwerks entsprechend angepasst wird.

Dieses kaskadierte Regelungskonzept ist in Abbildung 6 dargestellt. Die Aufteilung in mehrere gekoppelte Regelkreise erlaubt es, die komplexe nichtlineare Dynamik der Rakete in einfacher zu beherrschende Teilprobleme zu zerlegen. Insbesondere wird dadurch erreicht, dass Positionsabweichungen zunächst in gewünschte Orientierungen übersetzt werden, welche anschließend durch die Attitudenregelung umgesetzt werden.

Die Wahl mehrerer PID-Regler anstelle eines globalen Reglers bietet dabei mehrere Vorteile. Zum einen können die einzelnen Regelkreise unabhängig voneinander parametrisiert und abgestimmt werden, was die Stabilität und Verständlichkeit des Systems erhöht. Zum anderen ermöglicht die modulare Struktur eine einfache Erweiterung oder Anpassung einzelner Regelkreise, ohne die gesamte Steuerlogik verändern zu müssen.

Die konkrete Berechnung der Stellgröße erfolgt in jedem Regler nach dem bekannten PID-Prinzip, wobei der Fehler zwischen Soll- und Istwert bestimmt und daraus ein Steuerwert berechnet wird. Zur Vermeidung von Instabilitäten wird der Integratoranteil begrenzt (Anti-Windup) und die resultierende Stellgröße auf einen definierten Bereich beschränkt.

Es kann zwischen verschiedenen Kontrollmodi gewählt werden, je nachdem, welche Steuerungsstrategie für das aktuelle Szenario am besten geeignet ist. Die automatisierte Zielpunktregelung erlaubt dem Nutzer, im UI-Interface Reglerparameter zu definieren. Diese werden in einer eigenen Datenstruktur gespeichert, welche die Verstärkungsfaktoren des PID-Reglers sowie einen optionalen Bias-Term enthält. Dadurch können verschiedene Szenarien gespeichert und geladen werden. Die manuelle Steuerung hingegen ermöglicht es, mit den Maustasten die Schubstärke sowie die Schubrichtung

direkt zu steuern.

Um die verschiedenen Kontrollmodi zu unterstützen, ist die Steuerung abstrahiert, sodass die verschiedenen Regelkreise einfach implementiert und aktiviert werden können. Je nach gewähltem Modus bestimmen diese Regelkreise, welche Aspekte der Raketenbewegung aktiv geregelt werden und wie die Sollwerte generiert werden. Diese Abstraktion ermöglicht es, unterschiedliche Steuerungsstrategien innerhalb derselben Architektur zu realisieren, ohne die zugrunde liegenden Regler implementieren zu müssen.

Zur Analyse des Regelverhaltens werden die einzelnen Beiträge der P-, I- und D-Anteile separat berechnet und während der Simulation visualisiert. Dies erleichtert das Verständnis der Dynamik sowie die Parametrierung der einzelnen Regler erheblich.

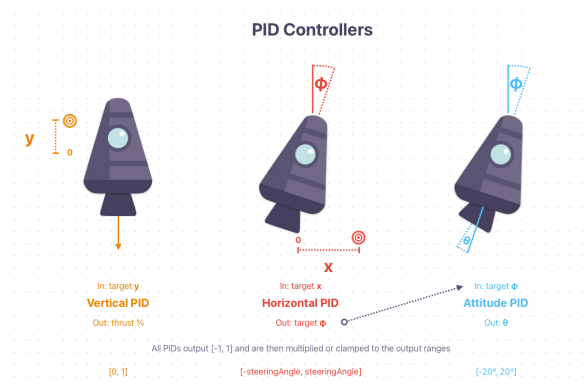


Abbildung 6: Regelungskonzept für die Schubvektorregelung

Tabelle 1: Verwendete Bibliotheken und deren Zweck. Die meisten Bibliotheken sind über Git-Submodule in das Projekt eingebunden. Beim Bauen der Anwendung werden die Bibliotheken eingebunden und kompiliert.

Bibliothek	Zweck
cereal	Serialisierung von C++-Objekten zur Speicherung
glfw	Erstellung von Fenstern sowie Verwaltung von Eingaben
imgui	UI-Bibliothek zur Erstellung der grafischen Benutzeroberfläche
miniaudio	Plattformübergreifende Bibliothek zur Wiedergabe von Audio
glew	OpenGL-Erweiterung
glm	Mathematikbibliothek für Vektoren, etc.
implot	Erweiterung von ImGui zur Darstellung von Plots
stb	Header-only-Bibliothek zum Laden von Texturen

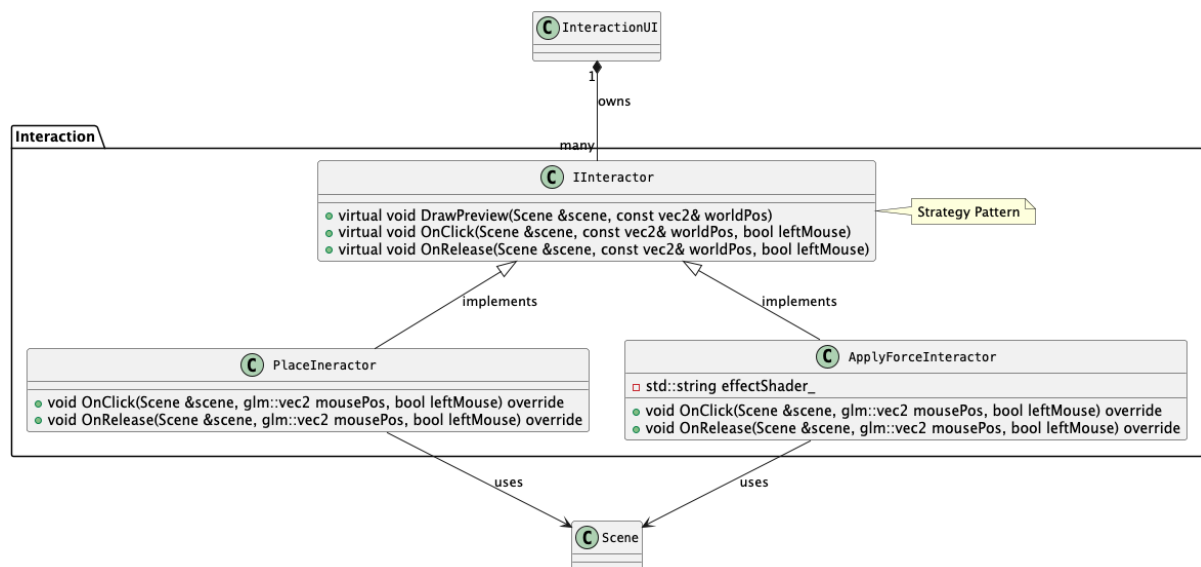


Abbildung 7: Überblick über die Struktur der physischen Interaktion mit der Simulation

```

classDiagram
    class SpriteVisual
    class LineRenderer
    class CircleRenderer
    class ShaderRenderer
    class ConstraintRenderer
    class SpriteRenderer
    class EngineLayer
    class TextureLoader
    class ShaderLoader
    class Serialiser
    class TextureInfo
    class PIDConfig
    class Scene

    SpriteVisual ..> TextureLoader : uses
    LineRenderer ..> TextureLoader : uses
    CircleRenderer ..> TextureLoader : uses
    ShaderRenderer ..> TextureLoader : uses
    ConstraintRenderer ..> TextureLoader : uses
    ShaderRenderer ..> ShaderLoader : uses
    ConstraintRenderer ..> ShaderLoader : uses
    SpriteRenderer ..> ShaderLoader : uses
    EngineLayer ..> Serialiser : uses
    TextureLoader ..> TextureInfo : uses
    Serialiser --> PIDConfig : saves
    Serialiser --> Scene : saves
  
```

TextureLoader

- Inline static unordered_map<string, TextureInfo> loadedTextures_
- static unsigned int whiteTex_
- static TextureInfo GetTexture(const string &path)
- static TextureInfo LoadTextureFromFile(const string &path)
- static unsigned int GetWhiteTexture()
- static void Clear()

ShaderLoader

- Inline static unordered_map<string, unsigned int> loadedShaders_
- static unsigned int LoadShader(const string &vertexPath, const string &fragmentPath)
- static void UnloadShader(unsigned int shaderProgram)
- static unsigned int CompileShader(unsigned int type, const char* source)
- static char* LoadSource(const string path)

Serialiser

- static string sceneFolder
- static void EnsureSceneFolderExists()
- static bool SaveScene(Scene &scene)
- static unique_ptr<Scene> LoadScene(const string &filePath)
- static bool SaveTempScene(Scene &scene)
- static unique_ptr<Scene> LoadTempScene()
- static bool SavePIDConfig(PIDConfig &pid, const filesystem::path &filePath)
- static unique_ptr<PIDConfig> LoadPIDConfig(const filesystem::path &filePath)

TextureInfo

- unsigned int id
- int width
- int height

Abbildung 9: Überblick über die Struktur der Serialisierung und Speicherung von Szenen und Reglereinstellungen